Lab 2 Booklet

UART, Interrupts, and Serial Protocols

Contents

Part "A": Direct Register Access **2**

1 Serial communication and the UART . . . 2
- 1.1 The UART . . . 2
- 1.2 The frame . . . 3

2 UART hardware on the STM32 . . . 4
- 2.1 UART peripherals . . . 4
- 2.2 The virtual COM port . . . 4
- 2.3 USART1 . . . 4
- 2.4 The registers of LPUART1 . . . 5
- 2.5 The device header . . . 6
- 2.6 Alternate functions . . . 7
- 2.7 The baud rate . . . 8
- 2.8 Transmission with the flag TXE . . . 8
- 2.9 Reception with the flags . . . 9

3 Interrupts and processor execution . . . 10
- 3.1 Interrupts . . . 10
- 3.2 The vector table and priorities . . . 10

4 Hardware random numbers . . . 11

Part "B": The HAL **12**

5 Messages and error detection . . . 12
- 5.1 Messages on a serial line . . . 12
- 5.2 The CRC . . . 13

6 Reception through the HAL . . . 15
- 6.1 Detecting a pause in reception . . . 15
- 6.2 Reception with the HAL . . . 16
- 6.3 Callbacks and priorities . . . 17

7 Hardware calculation and timing . . . 19
- 7.1 The CRC unit . . . 19
- 7.2 The cycle counter . . . 20

Appendix

A Modbus RTU . . . 22
- A.1 The message . . . 22
- A.2 Registers and function codes . . . 23
- A.3 Exception replies . . . 23
- A.4 A wrong CRC . . . 23

1 Serial communication and the UART

Serial communication transfers information one bit at a time. Dedicated hardware converts bytes into timed electrical signals and recovers bytes from incoming signals. The program exchanges bytes with the hardware; the hardware generates and samples the individual bits.

1.1 The UART

A universal asynchronous receiver-transmitter (UART) sends each character as a sequence of bits on one wire, the TX (transmit) line. It receives characters on a second wire, the RX (receive) line (Figure 1). The two sides have no common clock. Thus, they must use the same bit rate, the *baud rate*. The duration of one bit, the *bit time*, is 1 divided by the baud rate.

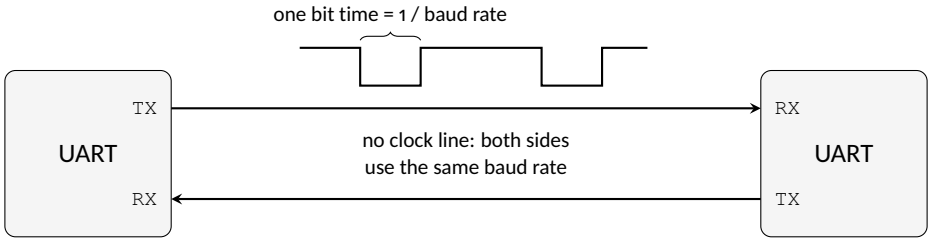


Figure 1: Two UARTs, with their two data lines.

A UART is one type of serial interface: it sends the bits one after the other. The serial peripheral interface (SPI) and the inter-integrated circuit bus (I2C) send a clock signal with the data. Such an interface is *synchronous*. A UART sends no clock signal. Thus, it is *asynchronous*: the receiver times the bits from the start of each character.

The TX line of each side goes to the RX line of the other side. Thus, the two sides can send at the same time.

The UART pins use the chip's logic levels: 0 V for 0, and V_{DD} for 1. On the NUCLEO-G491RE, V_{DD} is 3.3 V.

Many instruments and older computers use the RS-232 standard, with other voltages. In RS-232, a 1 is a negative voltage, from -3 to -15 V. A 0 is a positive voltage, from $+3$ to $+15$ V. Thus, an RS-232 line must never connect directly to a pin of the microcontroller. An external RS-232 transceiver converts between the line voltages and the logic levels of the chip.

1.2 The frame

Each character goes on the line as a *frame* (Figure 2):

- **Idle.** Between frames, the line is high (`1`).
- **Start bit.** The frame starts with one bit that is low (`0`). Its falling edge informs the receiver that a frame starts.
- **Data bits.** The 8 bits of the character follow, least significant bit first.
- **Stop bit.** The frame ends with one bit that is high (`1`).

This format is *8N1*: 8 data bits, no parity bit, and 1 stop bit. The time of one frame is the *character time*: 10 bit times in 8N1.

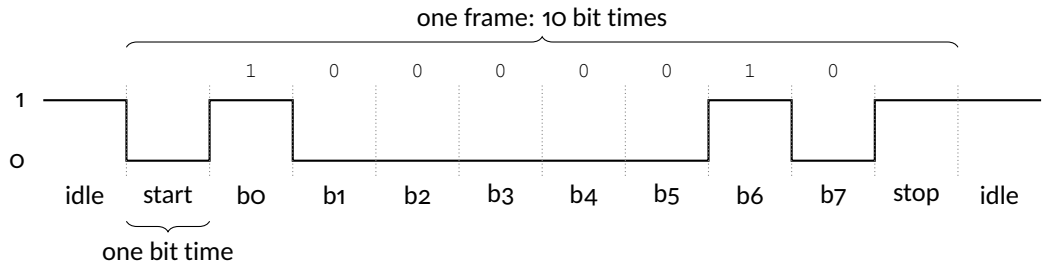


Figure 2: The frame of the character `A` (`0x41`) on the TX line.

The receiver starts to time the bits again at the falling edge of each start bit. Thus, a small error in the bit time affects only one frame. A UART can use other formats:

- 7 or 9 data bits, in place of 8.
- 2 stop bits, in place of 1.
- a parity bit after the data bits. With even parity, the sender makes the number of `1`s in the data bits and the parity bit even. The receiver can then find one wrong bit, but it cannot correct it.

The two sides must use the same format. If the receiver finds a `0` at the time of the stop bit, it sets the flag `FE`, framing error.

Each character in a text, including a space, has its own frame. For example, `HI ALL` uses six frames (Figure 3).

The next frame can start immediately after the previous stop bit. During a pause, TX stays high. The receiver recovers the characters in order. The program identifies words from the spaces.

The same UART can carry text or binary data. A byte's meaning comes from the application: the UART transports its bits without interpreting it as a letter, a command, or part of a number.

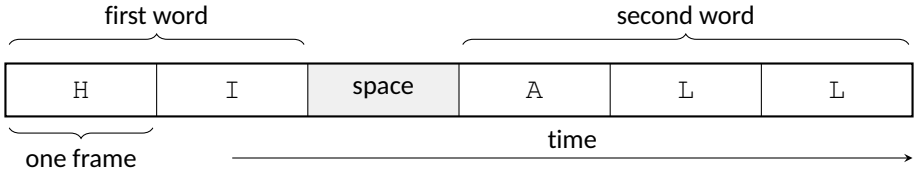


Figure 3: Two text words as a sequence of character frames.

2 UART hardware on the STM32

The UART signal format determines how bits travel on the wire. The microcontroller must also connect that signal to a physical pin and provide a clock at the required rate.

2.1 UART peripherals

The STM32G491RE has universal synchronous/asynchronous receiver-transmitter (USART) peripherals and a low-power UART, LPUART1. A USART operates as a UART in asynchronous mode. The peripheral sends and receives the bits in hardware. Separate SPI and I2C peripherals handle those interfaces.

2.2 The virtual COM port

The computer presents the USB connection as a serial port, called a *virtual COM port*. In Windows, the port has a name such as `COM3`. A terminal or PC program opens this port to send and receive bytes.

The NUCLEO-G491RE contains the STM32G491RE target chip and a separate ST-LINK chip. ST-LINK converts between USB and the 3.3 V signals of LPUART1 (Figure 4).

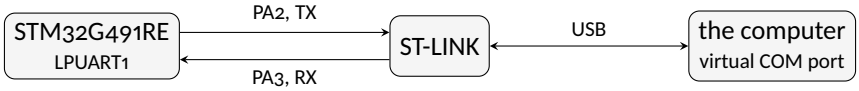


Figure 4: The virtual COM port.

The baud rate, number of data bits, parity, and number of stop bits must match the LPUART1 configuration. The PC program sets the UART format of ST-LINK. The target program configures LPUART1 separately.

2.3 USART1

PA2 and PA3 have no connection to a header pin. Thus, an oscilloscope cannot probe them. The TX pin of USART1, PC4, is on a header pin (Figure 5). Thus, an oscilloscope can show the frames that USART1 sends.

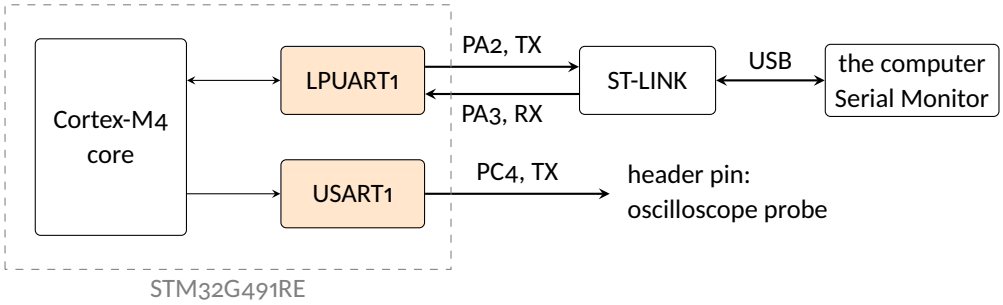


Figure 5: LPUART1 and USART1, and the path of each TX pin.

2.4 The registers of LPUART1

A program configures LPUART1, sends, and receives through the registers in Table 1 (Figure 6). The names `CR1`, `CR2`, and `CR3` mean control registers 1, 2, and 3.

Register	Function
`CR1`	Control: enables LPUART1 (`UE`), the transmitter (`TE`), the receiver (`RE`), and the receive interrupt (`RXNEIE`).
`CR2`	Control: selects stop bits, bit order, signal inversion, and TX/RX pin exchange.
`CR3`	Control: configures direct memory access (DMA), hardware flow control, half-duplex operation, and error interrupts.
`BRR`	Baud rate.
`ISR`	Status. Its flags give the state of LPUART1.
`ICR`	Flag clear. A `1` written to a bit clears the related flag of `ISR`.
`TDR`	The character to send.
`RDR`	The received character.

Table 1: Registers of LPUART1. USART1 has the same registers.

- **The clock.** A program enables the clock of each UART before it writes its registers: LPUART1 in `RCC_APB1ENR2`, and USART1 in `RCC_APB2ENR`.

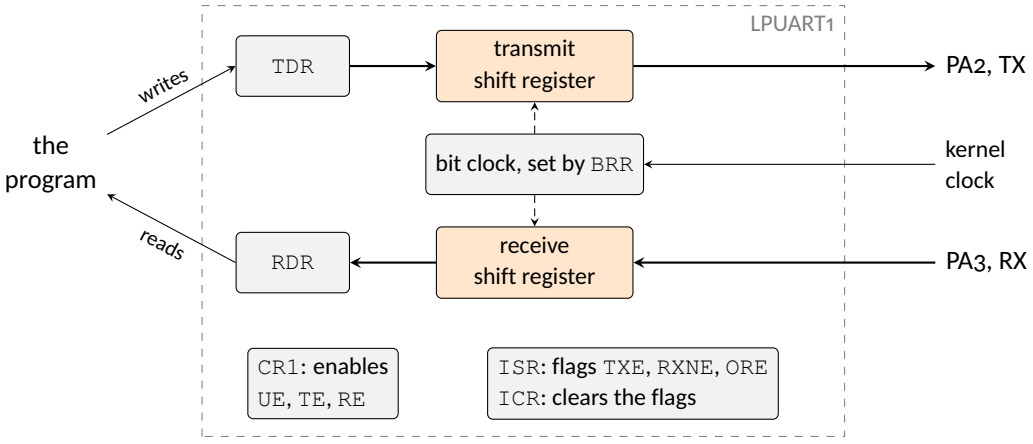


Figure 6: The registers in LPUART1 (simplified). The bit clock, set by `BRR`, times the bits of the two shift registers.

2.5 The device header

The device header `stm32g491xx.h` gives a name to each register, bit, and field of the chip. A program uses these names, and not the addresses. `main.h` includes the device header. Each name has one of three forms (Table 2):

Form	Example	What it names
Register	`LPUART1->BRR`	register `BRR` of LPUART1
Bit mask	`USART_CR1_TE`	bit `TE` of `CR1`
Field position	`GPIO_MODER_MODE4_Pos`	first bit of field `MODE4` of `MODER`

Table 2: The three forms of the names.

`LPUART1` is a pointer to a structure at the base address of LPUART1. Each member of the structure is one register, at its offset from the base. Thus, `LPUART1->BRR` is the register at the base address plus the offset of `BRR` (Figure 7).

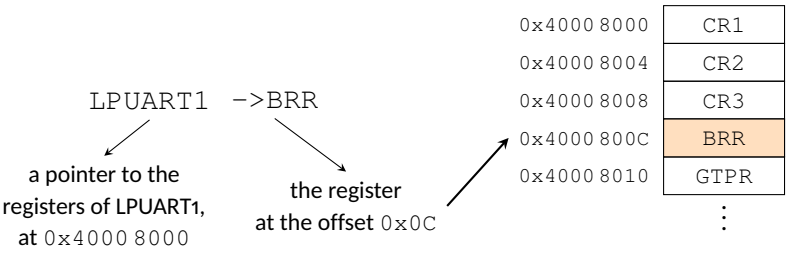


Figure 7: From a name of the device header to an address. `LPUART1->BRR` is the register at `0x4000 8000` + `0x0C`.

The register name and the bit mask have different functions:

- `LPUART1->CR1` selects the control register of LPUART1.
- `USART_CR1_TE` is a value with `1` at the `TE` bit and `0` at all other bits.

The device header uses the prefix `USART_` for these bit masks, also when the peripheral is LPUART1. The register name selects the peripheral to which the mask applies.

```
LPUART1->CR1 = USART_CR1_TE | USART_CR1_UE;      /* set TE and UE in CR1 */
if ((LPUART1->ISR & USART_ISR_TXE) != 0U) { }    /* test TXE in ISR */
```

- **First line.** The operator `|` combines the masks for `TE` and `UE`. The assignment sets these two bits to `1` and clears the other bits of `CR1`. This enables the transmitter and LPUART1.
- **Second line.** The operator `&` selects the `TXE` bit from `ISR`. The condition is true when `TXE` is `1`.

2.6 Alternate functions

With a GPIO output, the program sets the pin high or low through `ODR`. An *alternate function* (AF) connects a pin to a peripheral, so the peripheral can send or receive its signal through that pin. For example, LPUART1 sends its TX signal through PA2 and receives its RX signal through PA3. The transmitter produces the bits on PA2 in hardware, so the program does not need a GPIO write for each bit.

The chip has a limited number of pins. Several peripheral signals can share one pin, and the program selects the signal that the application needs (Figure 8). Each pin has a fixed set of available signals. One peripheral signal uses the pin at a time.

Two settings of port *A* connect PA2 and PA3 to LPUART1:

1. **The AF number.** Each pin has up to 16 alternate functions, with the numbers 0 to 15. The AF number of a peripheral can be different on each pin. A program sets the AF number first.
2. **The mode.** The program sets the field of the pin in `MODER` to `10`, alternate function.

The AF number of pin n is a 4-bit field (Table 3):

Register	In C	Pins	Field of pin n
`AFRL`	`AFR[0]`	0 to 7	bits $4n + 3$ to $4n$
`AFRH`	`AFR[1]`	8 to 15	bits $4(n - 8) + 3$ to $4(n - 8)$

Table 3: The alternate function registers of each port.

For example, this line sets pin 6 of port *B* to AF 5. It clears the field of the pin, then sets it:

```
GPIOB->AFR[0] = (GPIOB->AFR[0] & ~(0xFUL << 24)) | (5UL << 24);
```

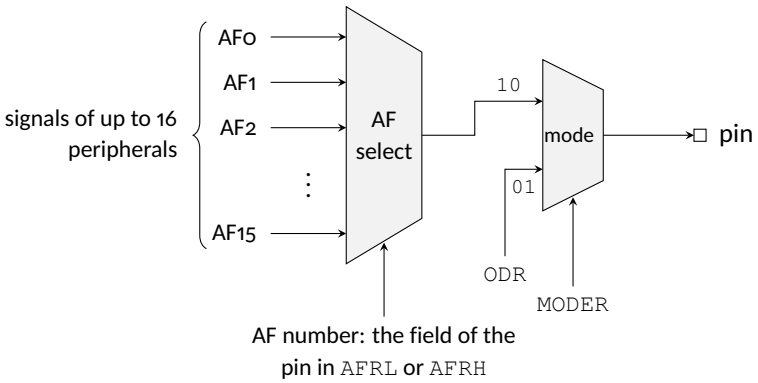


Figure 8: The output of a pin with alternate functions (simplified). The AF number selects one peripheral signal. In `MODER`, `10` connects this signal to the pin, and `01` connects `ODR`.

2.7 The baud rate

`BRR` sets the baud rate. It divides the clock of the UART, the *kernel clock* f_{ck}. With the clock settings that STM32CubeMX generates for this board, f_{ck} is PCLK, 170 MHz: the reset value `00` in `RCC_CCIPR` selects it. The two UARTs divide in different ways (Table 4):

UART	Baud rate	Limits
LPUART1	$256 f_{ck}/\mathtt{BRR}$	`BRR` is `0x300` or more. f_{ck} is 3 to 4096 times the baud rate.
USART1	$f_{ck}/\mathtt{BRR}$	`BRR` is 16 or more.

Table 4: The baud rate of each UART.

For example, USART1 at 9600 baud needs $\mathtt{BRR} = 170\,000\,000/9600 = 17\,708$, rounded. A program writes `BRR` only while `UE` is `0`.

The receiver has no clock signal from the sender. It finds the falling edge of the start bit, and then it reads each bit near its middle (Figure 9). LPUART1 takes one sample of each bit. USART1 takes 16 samples of each bit, or 8, and uses the samples near the middle. A difference between the baud rates of the two sides moves each later sample away from the middle of its bit. Thus, the difference must stay small: for USART1, with 16 samples and 8 data bits, the receiver permits up to 3.75 % in total.

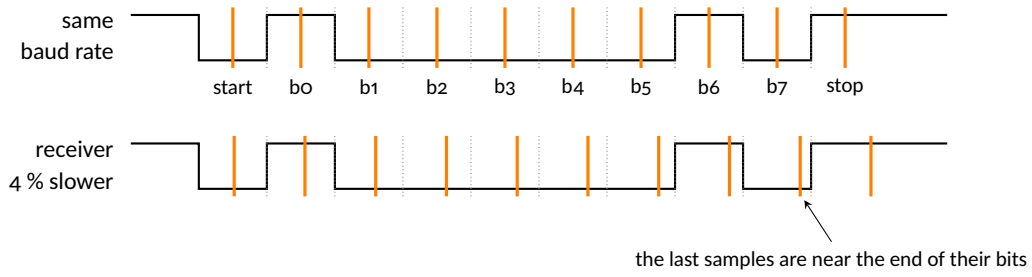


Figure 9: Where a receiver samples the bits of a frame (orange). Top: the same baud rate on the two sides. Bottom: a receiver 4 % slower than the sender. Each sample comes later than the middle of its bit.

2.8 Transmission with the flag TXE

A program sends characters with the flag `TXE` of `ISR`, transmit data register empty (Figure 10).

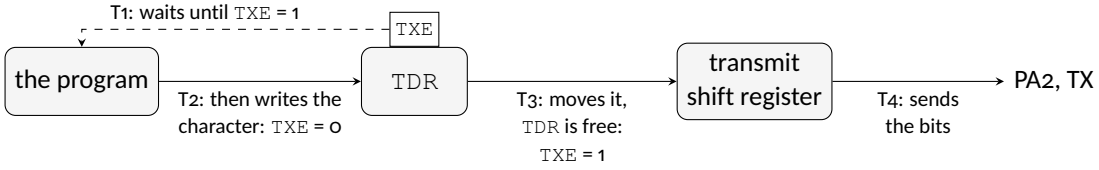


Figure 10: A transmission through LPUART1.

`ISR` has a second flag for transmission: `TC`, transmission complete. `TXE` becomes `1` when `TDR` is free, while the shift register still sends the character. `TC` becomes `1` only after the stop bit of the last character, when the line is idle (Figure 11). Thus, a program waits for `TC` before it switches off the UART.

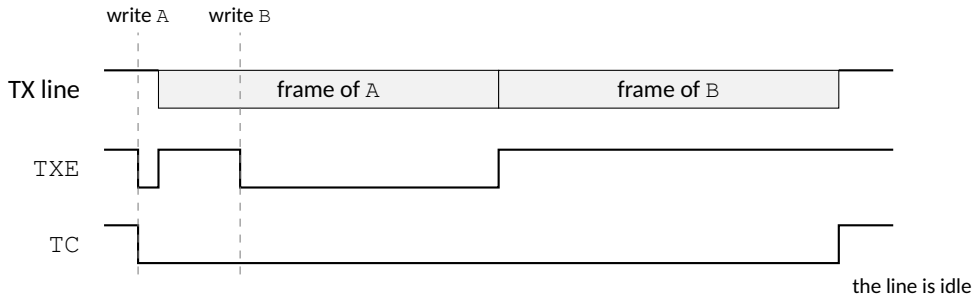


Figure 11: TXE and TC when a program sends the characters A and B.

2.9 Reception with the flags

A program receives characters with two flags of ISR, RXNE and ORE (Table 5, Figure 12).

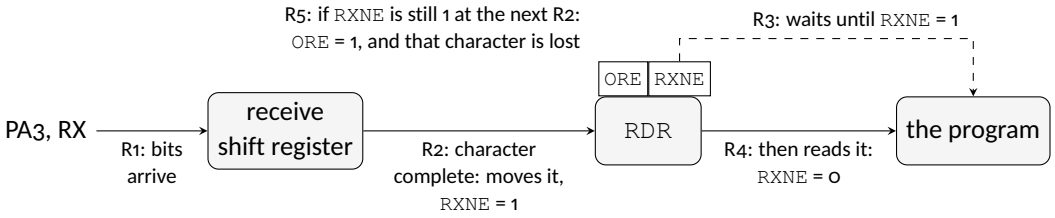


Figure 12: A reception through LPUART1.

Flag	It becomes 1 when	It becomes 0 when
RXNE read data register not empty	RDR holds a received character	the program reads RDR
ORE overrun error	a character arrives while RXNE is 1	the program writes 1 to ORECF in ICR

Table 5: The two flags of ISR of a reception.

Overrun. When ORE becomes 1, the new character is lost. Each later character is lost too, until the program clears ORE.

A UART can reduce the risk of an overrun in three ways:

- **A FIFO.** LPUART1 and USART1 each have a receive FIFO (first in, first out), a small buffer of characters in front of RDR. It is off after a reset.
- **Flow control.** With two more lines, RTS and CTS, the receiver can stop the sender.
- **DMA.** A direct memory access (DMA) controller copies each character from RDR into memory, with no instruction of the processor.

A program that repeatedly tests a status flag uses *polling*. Its ability to receive each character depends on how often it reaches that test.

3 Interrupts and processor execution

3.1 Interrupts

A loop that tests `RXNE` finds a character only when it reaches the test. While the loop executes a long operation, the next characters are lost. An interrupt prevents this loss: the hardware stops the loop when a character arrives (Figure 13):

1. LPUART1 sets `RXNE` and requests the interrupt.
2. The processor stops the loop after the current instruction, and saves some core registers on the stack.
3. The processor runs the *handler* of the interrupt. The handler reads `RDR`.
4. The processor restores the registers. The loop continues.

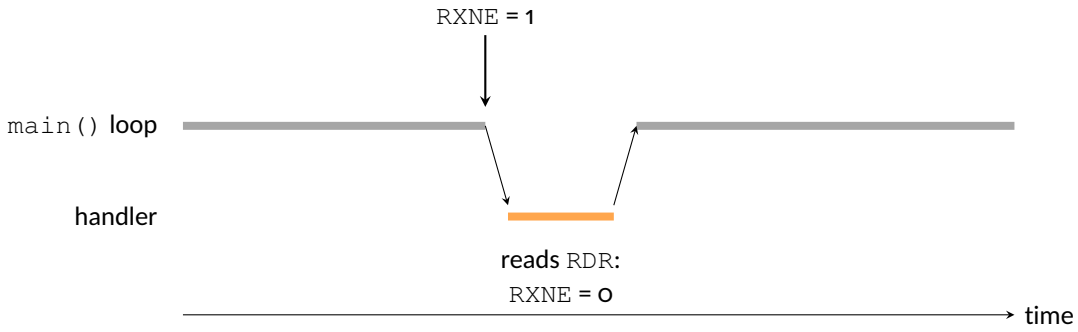


Figure 13: An interrupt.

Before the handler returns, it must clear each flag that requests the interrupt. Otherwise, the request stays active: the handler runs repeatedly, and the main loop no longer runs:

- A read of `RDR` clears `RXNE`.
- `ORE` also requests the interrupt, and a read of `RDR` does not clear it. To clear `ORE`, a program writes `USART_ICR_ORECF` to `ICR`.

3.2 The vector table and priorities

The NVIC is the nested vectored interrupt controller of the Cortex-M4.

The *vector table* is at the start of flash, `0x0800 0000`. It is a list of 32-bit addresses (Figure 14). The first word is the start value of the stack pointer, and the second word is the address of `Reset_Handler`. Each later word is the address of the handler of one exception or interrupt. When an interrupt starts, the processor reads the address of its handler from the table.

The startup file gives each handler a default, `Default_Handler`, an endless loop. The default is *weak*: a C function with the same name replaces it. Thus, a handler with a wrong name does not replace the default, and the first interrupt stops the program in the endless loop.

The NVIC gives each interrupt a priority, from 0 to 15. A smaller number is a higher priority. An interrupt of a higher priority can stop the handler of an interrupt of a lower priority. While a handler runs, the interrupts of the same or a lower priority wait. Thus, a handler must be short.

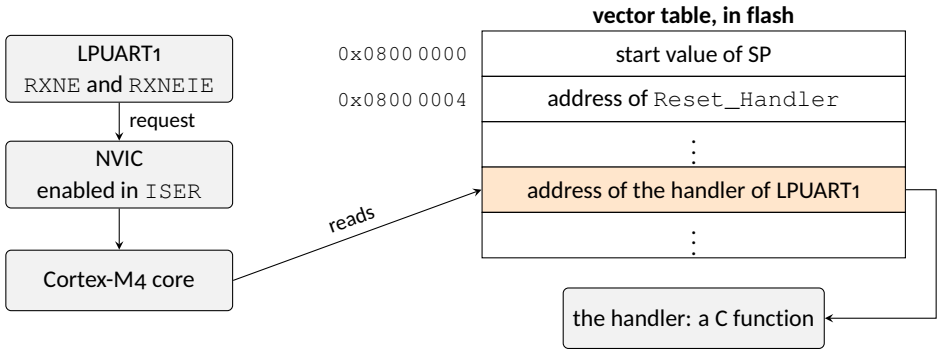


Figure 14: From a peripheral to its handler (simplified).

An interrupt needs three items (Table 6):

Item	What the program does
Enable in LPUART1	Sets `RXNEIE` in `CR1`.
Enable in the NVIC	The program writes a `1` to the bit of the interrupt in `NVIC->ISER`. It uses `=`, not `|=`: a `0` bit has no effect, so the store does not disable other interrupts.
The handler	Defines a C function with the name of the interrupt in `startup_stm32g491xx.s`.

Table 6: The three items of an interrupt. The vector table gives the number m of each interrupt.

Interrupt m is bit $m \bmod 32$ of `NVIC->ISER[`k`]`, where k is $m/32$, rounded down. For example, interrupt 40 is bit 8 of `NVIC->ISER[1]`:

```
NVIC->ISER[1] = 1UL << 8;   /* enable interrupt 40 */
```

4 Hardware random numbers

Software can produce a *pseudo-random* sequence with a formula. The sequence starts from an initial state, the *seed*. The same seed produces the same sequence. The C function `rand()` uses this method.

The STM32G491RE has a *true random number generator* (RNG). It obtains unpredictable bits from electrical noise in ring oscillators. A *ring oscillator* connects an odd number of inverters in a loop. Each inverter introduces a delay. A change travels around the loop, so the output changes repeatedly between `0` and `1`.

Electrical noise causes small, unpredictable changes in the times of these transitions. This variation is *jitter*. A separate clock samples the signal. If a rising edge occurs before the sample, the sampled bit is `1`. If the edge occurs after it, the sampled bit is `0` (Figure 15).

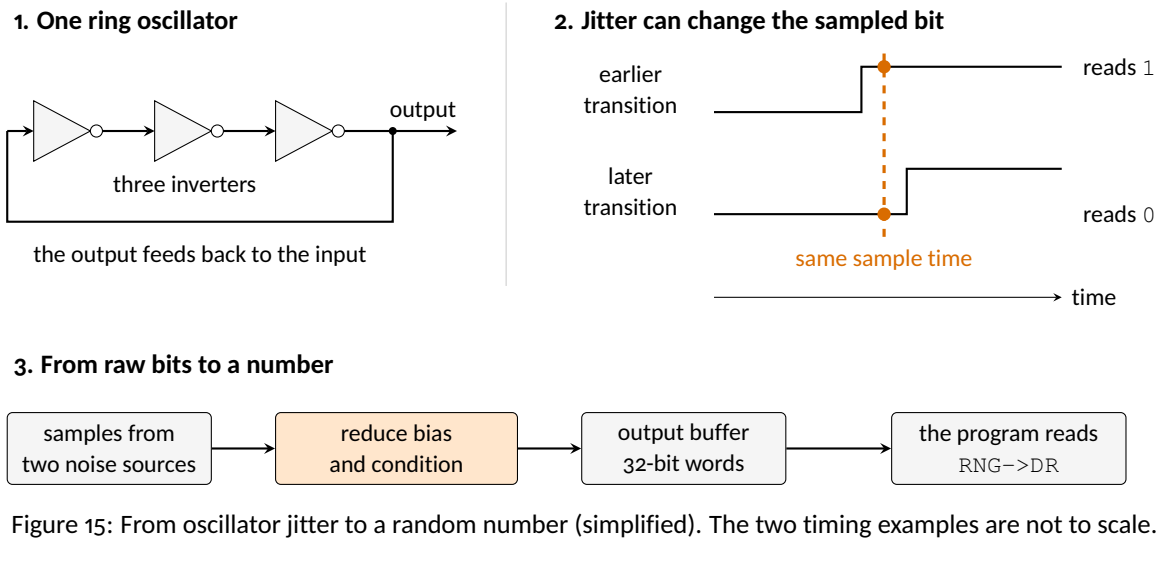


Figure 15: From oscillator jitter to a random number (simplified). The two timing examples are not to scale.

Each of the RNG's two noise sources combines three oscillator outputs with XOR. The sample clock can come from HSI48, the internal 48 MHz oscillator.

Raw bits can have unequal probabilities of `0` and `1`. This is a *bias*. Post-processing reduces the bias. The conditioning circuit combines samples into 128-bit blocks. An output buffer holds four 32-bit words. The AHB clock operates the conditioning circuit and the register interface.

A program checks `DRDY` and the error flags in `RNG->SR` before it reads `RNG->DR`. Hardware tests detect faults in the noise sources and the sample clock. A seed error means that the noise source failed a health test. The program must reject the affected data and restore the RNG before it uses further numbers.

Random numbers can repeat. A reset does not guarantee a different next number.

5 Messages and error detection

A UART recovers a sequence of characters. The application still needs rules for grouping those characters into messages and deciding whether a received message can be used. These rules belong to a protocol.

5.1 Messages on a serial line

A *message* is a group of bytes with one function, for example a request to a device, or its reply. A *protocol* defines the rules for exchanging messages. A receiver of a stream of bytes must find where each message starts and ends. Protocols use one of three methods (Figure 16):

- **A silence.** The line stays idle for a set time after each message. Thus, the sender must send each message with no pause. Modbus RTU, a protocol of industrial devices, uses this method.
- **An end mark.** A special byte ends each message, for example `\n` at the end of a line of text. A data byte that is equal to the end mark must then go in a different form. SLIP (serial line Internet protocol) and COBS (consistent overhead byte stuffing) are protocols of this type.
- **A length.** The first bytes of each message give its length. If one byte is lost, the receiver reads a data byte as the next length. Then it finds a wrong start for each next message.

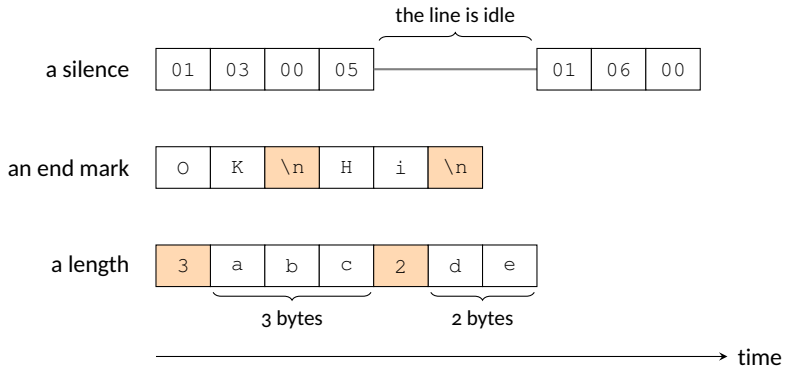


Figure 16: Three methods to find the end of each message in a stream of bytes. Orange: the end marks and the lengths.

5.2 The CRC

Noise on a line can change bits. A *cyclic redundancy check* (CRC) finds such errors. The sender calculates the CRC from the bytes of the message, and sends it after them. The receiver calculates the CRC from the message bytes before the received CRC. If the two CRCs are different, the message has an error (Figure 17).

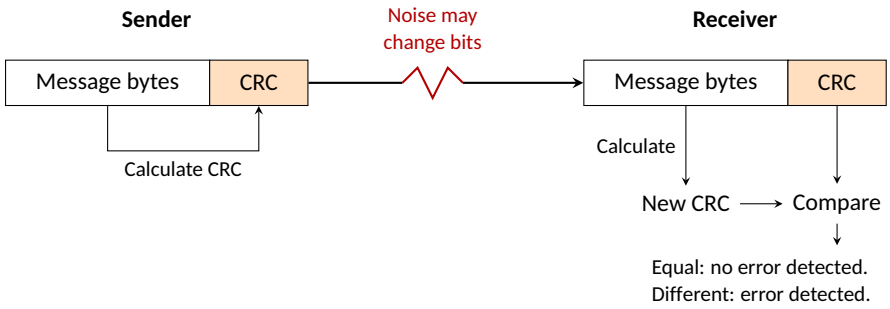


Figure 17: The sender appends a CRC. The receiver calculates a new CRC from the received message bytes and compares the two values.

The calculation divides the bits of the message by a fixed number, the *polynomial*. The division uses exclusive OR (XOR) in place of subtraction, with no carry. The CRC is the remainder.

Worked example: a 3-bit CRC

The message is `1101`, which represents $M(x) = x^3 + x^2 + 1$. The divisor is `1011`, which represents $G(x) = x^3 + x + 1$. Each bit is a coefficient, from the highest power of x to x^0. The divisor has degree 3, so the CRC has 3 bits. The initial CRC is `000`; the most significant bit is processed first, with no final XOR.

1. Append zeros. The sender appends three zeros to the message, giving `1101000`. This multiplies $M(x)$ by x^3 and leaves space for the 3-bit remainder.

2. Divide with XOR. The sender aligns the leftmost `1` of the divisor with the leftmost `1` of the current result, then applies XOR. For example, $1 \oplus 1 = 0$ and $0 \oplus 1 = 1$; there is no borrowing. Each row shows one alignment and XOR:

$$1101000 \oplus 1011000 = 0110000$$
$$0110000 \oplus 0101100 = 0011100$$
$$0011100 \oplus 0010110 = 0001010$$
$$0001010 \oplus 0001011 = 0000001.$$

The remaining polynomial is 1, with degree below 3, so the division stops. The remainder, written with three bits, is `001`.

3. Send and check. The sender replaces the appended zeros with the CRC and sends:

$$\underbrace{1101}_{\text{message}}\ \underbrace{001}_{\text{CRC}}.$$

If these bits arrive unchanged, the receiver calculates the CRC of `1101` by the same steps and obtains `001`. It matches the received CRC, so no error is detected.

4. Detect a changed bit. Suppose noise changes the second message bit, from the left, giving `1001`; the received CRC remains `001`. The receiver appends three zeros to `1001` and calculates:

$$1001000 \oplus 1011000 = 0010000$$
$$0010000 \oplus 0010110 = 0000110.$$

The new CRC is `110`, which differs from the received `001`. The receiver detects an error.

A CRC standard sets five parameters (Table 7):

- the number of bits of the CRC.
- the polynomial. For example, `0x8005` is $x^{16} + x^{15} + x^2 + 1$: the highest term is not written.
- the start value of the CRC.
- the sequence of the bits of each byte: bit 0 first, as on a UART line, or bit 7 first.
- a final value, for XOR with the result.

Standard	Bits	Polynomial	Start	Bit 0 first	Final XOR	Used in
CRC-16/MODBUS	16	`0x8005`	`0xFFFF`	yes	`0x0000`	Modbus RTU
CRC-16/XMODEM	16	`0x1021`	`0x0000`	no	`0x0000`	XMODEM
CRC-32	32	`0x04C11DB7`	`0xFFFFFFFF`	yes	`0xFFFFFFFF`	Ethernet, ZIP

Table 7: Three CRC standards. The CRC of the ASCII text `123456789` checks a program: `0x4B37`, `0x31C3`, and `0xCBF43926`.

When bit 0 goes first, a program uses the polynomial with its bits in the opposite sequence: `0xA001` for

`0x8005`. For example, a program calculates CRC-16/MODBUS byte by byte. It adds each byte to the CRC with XOR. Then, 8 times, it shifts the CRC right by one bit. If the shift removes a `1`, the program applies XOR with `0xA001`.

A CRC of 16 bits finds each error of one bit, and each error in an odd number of bits. It also finds each burst of errors of 16 bits or fewer. Of the other errors, it misses about 1 in 65536.

A receiver ignores a message with a wrong CRC. In a protocol of requests and replies, the receiver then sends no reply. The sender waits for a limited time, the *time limit*. With no reply in this time, it sends the request again (Figure 18).

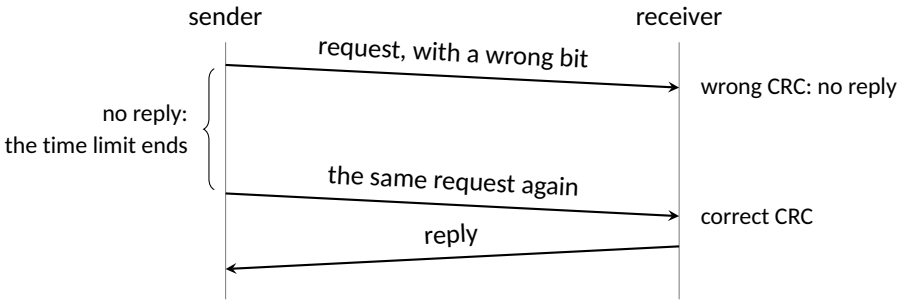


Figure 18: A request with a wrong bit, and the same request again after the time limit.

6 Reception through the HAL

The hardware abstraction layer (HAL) supplies library functions that manage peripheral registers and interrupt handling. Reception into a buffer combines the UART's pause detection with software that passes the received bytes to the application.

6.1 Detecting a pause in reception

The UART can notify a program when incoming data pauses. The `IDLE` flag provides this notification, allowing the program to process the bytes received so far.

After one or more received characters, the UART sets `IDLE` in `ISR` when the RX line stays high for one character time. Setting `IDLEIE` in `CR1` enables this event to request an interrupt (Figure 19).

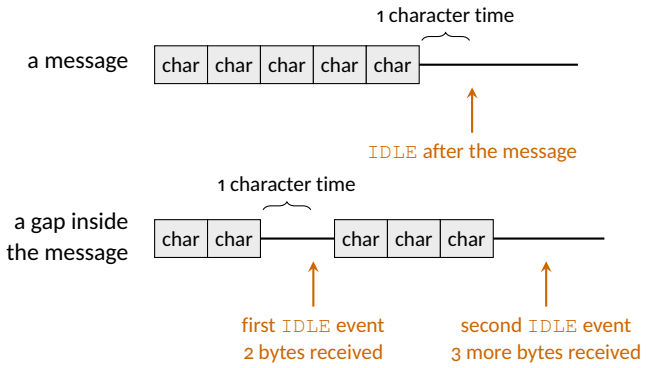


Figure 19: The UART detects pauses after received bytes. Top: a pause after a complete message. Bottom: an internal pause triggers an earlier `IDLE` event.

In the lower row, all five characters belong to one protocol message. The pause after the second character lasts more than one character time, so `IDLE` is set before the third character arrives. The last three characters are followed by another pause and another `IDLE` event. If each event ends a reception, the program receives one message in two parts. The protocol's framing rules determine when the complete message has arrived.

6.2 Reception with the HAL

The hardware abstraction layer (HAL) uses `IDLE` to end a reception into a buffer. Each UART function of the HAL takes the address of `hlpuart1`, the *handle* of LPUART1. The handle is a structure, in the generated code, with the configuration and state of LPUART1.

A *callback* is a function supplied by the program for the HAL to call when an event occurs. The program defines the function's body; the HAL determines when to call it.

The handler of the interrupt, `LPUART1_IRQHandler()` in `stm32g4xx_it.c`, calls `HAL_UART_IRQHandler()`. This HAL function reads the flags of LPUART1, and it calls the callback (Figure 20). The HAL declares each callback with `__weak`: an empty default. A function with the same name in the program replaces the default when the linker makes the program.

Table 8 summarizes transmission, reception, and callbacks in the HAL.

Function	What it does
`HAL_UART_Transmit()`	Sends a buffer. It returns after the last character, or when its time limit ends.
`HAL_UARTEx_ ReceiveToIdle_IT()`	Starts a reception by interrupt into a buffer, and returns immediately. The reception ends when the line is idle, or when the buffer is full.
`HAL_UARTEx_ RxEventCallback()`	A program writes it. The HAL calls it from the interrupt handler at the end of a reception, with the number of bytes.
`HAL_UART_ErrorCallback()`	A program writes it. The HAL calls it after an error, for example an overrun.

Table 8: Four UART functions of the HAL.

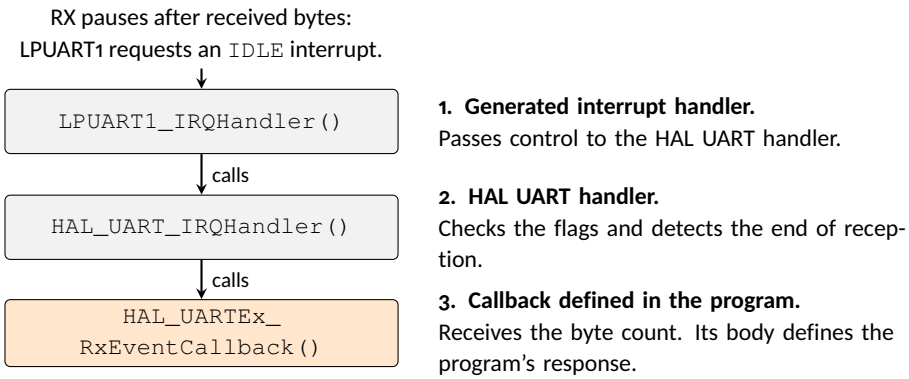


Figure 20: The HAL calls the program's callback when reception ends. All three functions run as part of the UART interrupt handler.

To receive messages by interrupt:

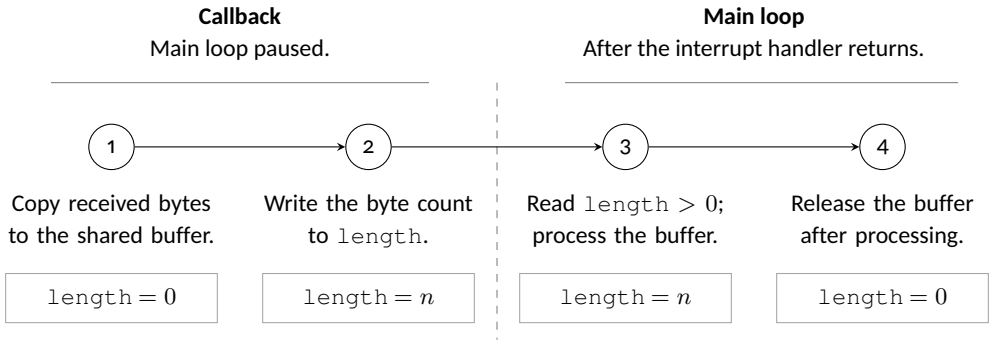
1. `HAL_UARTEx_ReceiveToIdle_IT(&hlpuart1, buf, size)` starts the reception of `size` bytes, at most, into `buf`.
2. At the end of the reception, the HAL calls `HAL_UARTEx_RxEventCallback()` with the number of bytes.
3. The callback copies the received bytes to a separate buffer. Then it calls `HAL_UARTEx_ ReceiveToIdle_IT()` again for the next message.
4. After an error, the HAL stops the reception and calls `HAL_UART_ErrorCallback()`. This callback also starts the reception again.

6.3 Callbacks and priorities

The HAL calls a callback to notify the program of an event. For example, when a UART reception ends, the HAL calls `HAL_UARTEx_RxEventCallback()` with the number of received bytes. The program defines what this function does with those bytes.

During reception by interrupt, this callback runs inside the UART interrupt handler. The loop of `main()` is paused until the handler returns. To keep this pause short, the callback copies the bytes to a separate buffer and records their number. After the handler returns, the loop resumes and uses that number to detect and process the received data (Figure 21).

A variable that the callback writes and the loop reads must be `volatile`. Then the loop reads it from memory on each pass. A new message can arrive before the loop processes the previous message. Then the callback must discard the new message, or keep it in a second buffer.



n: number of received bytes. Clearing `length` frees the buffer; it does not erase the bytes.

Figure 21: The shared length signals when data are ready. The callback sets it after copying the bytes; the main loop clears it after processing them.

Interrupts with the same priority cannot interrupt each other. A smaller priority number means a higher priority.

The generated code assigns priority 0 to both LPUART1 and SysTick. While a UART callback runs:

- SysTick cannot run, so the tick count returned by `HAL_GetTick()` does not increase.
- `HAL_Delay()` waits for that count to increase, so calling it inside the callback prevents the callback from finishing.

The program leaves SysTick at priority 0 and sets LPUART1 to priority 1:

```
HAL_NVIC_SetPriority(LPUART1_IRQn, 1, 0);
```

SysTick can then interrupt the UART handler and update the tick count (Figure 22). After SysTick returns, the UART handler resumes. HAL time limits can therefore expire while the callback runs.

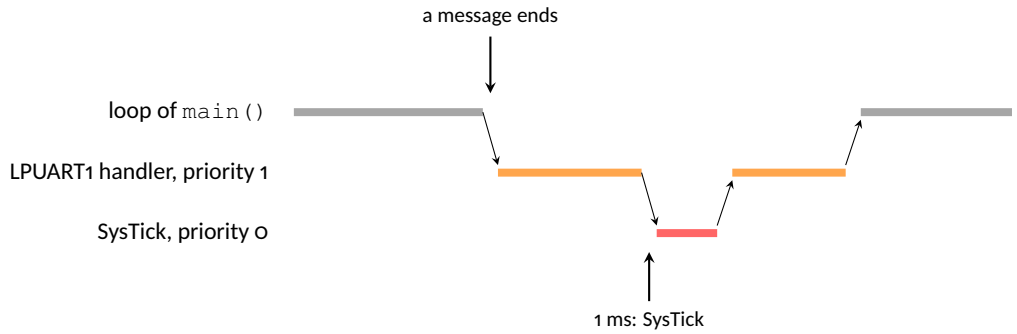


Figure 22: SysTick, with priority 0, stops the handler of LPUART1, with priority 1. Thus, the tick count increases while a callback runs.

7 Hardware calculation and timing

A hardware calculation unit can replace a sequence of processor instructions. Comparing the complete software and hardware operations requires measuring both the calculation and the instructions that transfer its input and result.

7.1 The CRC unit

In software, the CRC of each byte takes 8 steps. The STM32G491RE has a *CRC unit*: a peripheral that calculates a CRC in hardware. A program writes the bytes to its data register, `DR`, and then reads the CRC from `DR` (Figure 23). The other registers set the CRC (Table 9).

The CRC unit takes bit 7 of each byte first. Some standards take bit 0 first, for example CRC-16/MODBUS and CRC-32. For these standards, the CRC unit must reverse the bit sequence of each input byte. It must also reverse the bit sequence of the CRC result. The CRC unit has no final XOR: a program applies it to the result.

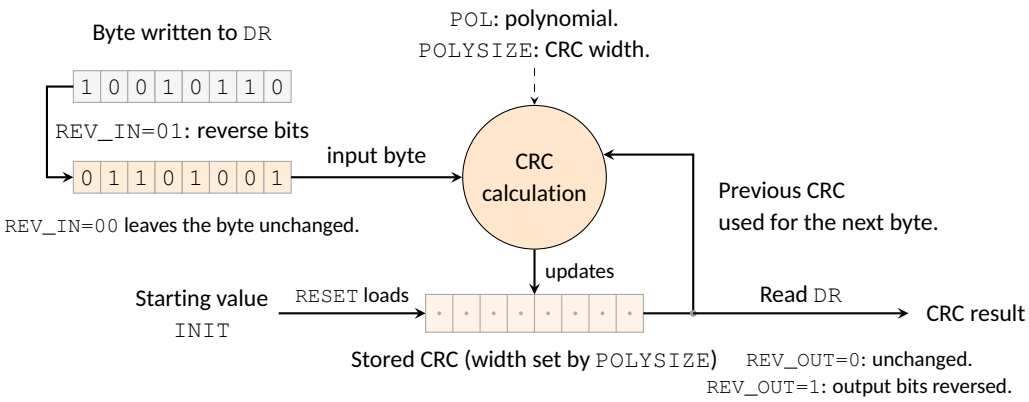


Figure 23: The CRC unit combines each input byte with the stored CRC. The feedback path reuses that value for the next byte. The register strip represents a selectable CRC width.

Item	Function
`DR`, offset `0x00`	A write gives data to the calculation: a write of 32 bits gives 4 bytes, and a byte access gives one byte. A read gives the CRC. A CRC of 16 bits is in bits 15 to 0.
`CR`, offset `0x08`: `RESET`, bit 0	Starts a new CRC: `DR` gets the value of `INIT`. The hardware clears the bit.
`CR`: `POLYSIZE`, bits 4 to 3	The size of the polynomial: `00` 32 bits, `01` 16 bits, `10` 8 bits, `11` 7 bits.
`CR`: `REV_IN`, bits 6 to 5	The reversal of the input data: `00` none, `01` in each byte.
`CR`: `REV_OUT`, bit 7	`1`: the reversal of the CRC.
`INIT`, offset `0x10`	The start value.
`POL`, offset `0x14`	The polynomial. Its reset value, `0x04C1 1DB7`, is the polynomial of a CRC of 32 bits.
Calculation time	The unit needs 1 AHB clock cycle for each input byte.

Table 9: The registers and calculation time of the CRC unit. Its clock enable is `CRCEN`, bit 12 of `RCC_AHB1ENR`.

In the device header, the CRC unit is `CRC`, for example `CRC->POL`. A bit name of a field ends with the number of the bit in the field. For example, `CRC_CR_POLYSIZE_0` is bit 0 of `POLYSIZE`, which is bit 3 of `CR`. With the HAL, `__HAL_RCC_CRC_CLK_ENABLE()` sets `CRCEN`.

The chip has more hardware units of this type. Each unit calculates its result in fewer clock cycles than a program:

- the RNG: random numbers.
- CORDIC: trigonometric functions, for example sine and cosine.
- FMAC: digital filters.

A unit of this type also frees the processor for other operations.

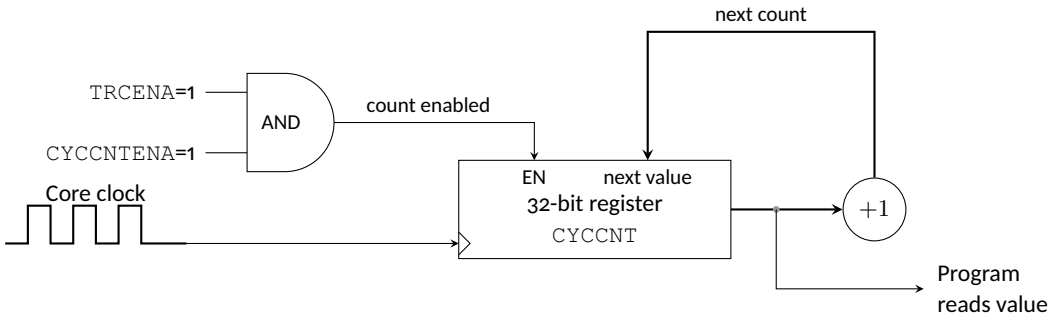
7.2 The cycle counter

To measure the time of an operation in clock cycles, a program reads the cycle counter `DWT->CYCCNT` before and after the operation. It is a 32-bit counter of the data watchpoint and trace unit (DWT) of the Cortex-M4. Two bits start it (Table 10):

Bit	Register	CMSIS names
`TRCENA`	`DEMCR`	`CoreDebug->DEMCR`, `CoreDebug_DEMCR_TRCENA_Msk`
`CYCCNTENA`	`DWT_CTRL`	`DWT->CTRL`, `DWT_CTRL_CYCCNTENA_Msk`

Table 10: The two bits that start the cycle counter. A program sets `TRCENA` first.

`TRCENA` enables the DWT; `CYCCNTENA` enables its cycle counter. While the processor runs with both bits set, hardware adds 1 to the counter on each core clock cycle (Figure 24). After `0xFFFF FFFF`, the next increment gives 0. Reading `CYCCNT` copies its current value; counting continues.



Two readings: $\underbrace{120}_{\text{start}}$ and $\underbrace{127}_{\text{end}}$ give $127 - 120 = 7$ elapsed clock cycles.

Figure 24: Functional circuit model of the cycle counter. Each enabled clock cycle stores the previous value plus 1. A read copies the value without resetting the counter.

```
uint32_t start = DWT->CYCCNT;            /* before the operation */
/* the operation */
uint32_t cycles = DWT->CYCCNT - start;   /* after the operation */
```

The subtraction is unsigned, as for the counter of `HAL_GetTick()`. At 170 MHz, 170 cycles are 1 μs. The result includes all instructions between the two counter reads. For a peripheral operation, this includes the instructions that transfer its data.

A program can measure time in four ways:

- `HAL_GetTick()`: in milliseconds, from SysTick.
- the cycle counter: in clock cycles, 5.9 ns each at 170 MHz.
- a hardware timer of the chip: at a rate that the program selects.
- a pin and an oscilloscope: the program sets a pin at the start of the operation and clears it at the end. The oscilloscope measures the pulse.

An interrupt that occurs during the operation adds the time of its handler to the measurement.

Appendix A: Modbus RTU

Modbus is a protocol for the control of industrial devices, for example sensors, motor drives, and energy meters. Modbus RTU is its form for serial lines. One device, the *master*, sends *requests*. The other devices are *slaves*.

Each slave has an address from 1 to 247. A slave sends a *reply* only to a request with its address (Figure 25). A request to the address 0 goes to all the slaves, and no slave replies to it. On a long line, the devices share a pair of wires, with the voltages of the RS-485 standard.

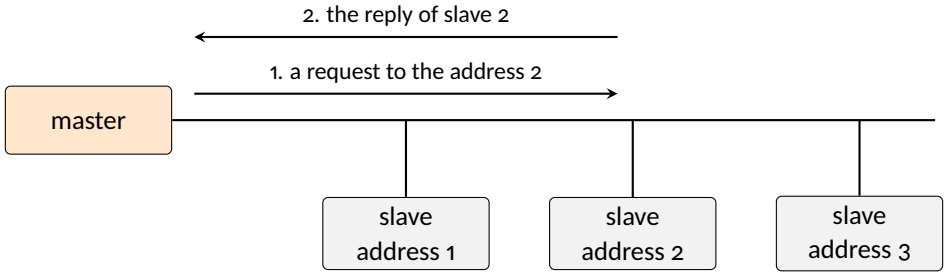


Figure 25: A master and three slaves on one line.

A.1 The message

Each message is a sequence of bytes (Figure 26):

- the address of the slave: 1 byte.
- the *function code*: 1 byte. It gives the operation that the slave must do.
- the data: 0 to 252 bytes. A 16-bit value goes as two bytes, the high byte first.
- the CRC: 2 bytes, the low byte first: CRC-16/MODBUS (Table 7).

After each message, the line stays idle for at least 3.5 character times. Inside a message, a gap between two characters must be shorter than 1.5 character times. Thus, the receiver finds the end of each message from the idle line.

A Modbus slave keeps its data in 16-bit *holding registers*, numbered from 0. The master reads and writes them. The *register map* of a device gives the content of each register. For example, one register of an energy meter can hold the voltage.

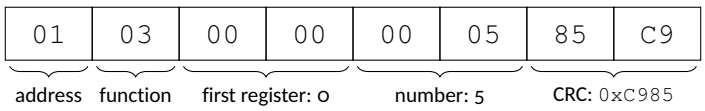


Figure 26: A request of Modbus RTU, in hexadecimal: read registers 0 to 4 of slave 1 (function `0x03`). The CRC is `0xC985`: its low byte goes first.

A Modbus message can have a gap of 1 to 1.5 character times inside it. Such a gap divides the message into two receptions (Detecting a pause in reception, Figure 19). Each part then has a wrong CRC.

A.2 Registers and function codes

The function code of a request gives the operation on the registers (Table 11, Figure 27). Other function codes read and write single bits, and write many registers in one request.

Code	Function	Data of the request	Data of the reply
`0x03`	Read holding registers	The address of the first register, and the number of registers, from 1 to 125.	The number of data bytes, in one byte. Then the value of each requested register.
`0x06`	Write single register	The register, and its new value.	A copy of the request.

Table 11: Two function codes of Modbus. Each register number and each value has 16 bits, and goes high byte first.



Figure 27: A read of registers 1 and 2 of slave 1, in hexadecimal. The reply holds 4 data bytes: the values 10 and 3.

A.3 Exception replies

A slave sends an *exception reply* when it receives a correct request, but cannot do the operation. Its function code is the function code of the request with bit 7 set: `0x83` for `0x03`. One byte follows, the exception code (Table 12). For example, `01 83 02 C0 F1` informs the master that the read asks for a register that slave 1 does not have. The master then knows that the slave received the request, but did not do it.

Code	Name	The slave sends it when
`01`	Illegal function	it does not have the function of the request.
`02`	Illegal data address	a register of the request is not in its register map.
`03`	Illegal data value	a value of the request is not permitted, for example a read of 0 registers.

Table 12: Three exception codes of Modbus.

A.4 A wrong CRC

A Modbus slave does not reply to a message with a wrong CRC: the wrong bit can be in the address. The master sends the request again when its time limit ends (Figure 18).